

Sistema Wearable de Monitoreo de Actividades de la Vida Diaria basado en ESP32: Arquitectura Distribuida con ESP-NOW, IMU, Pantalla Táctil y Registro en Tarjeta SD

Carlos Daniel Cuellar Antury

Departamento de Ingeniería Electrónica y Telecomunicaciones, Universidad del Cauca, Popayán, Colombia

Repositorio: <https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria>

Resumen — Este artículo describe el diseño e implementación de un sistema embebido distribuido para el monitoreo de Actividades de la Vida Diaria (AVD), desarrollado como proyecto académico en la Universidad del Cauca. El sistema adopta una arquitectura de tres nodos ESP32 interconectados mediante el protocolo ESP-NOW: un nodo maestro que integra un sensor de frecuencia cardíaca (ADC), reloj de tiempo real DS3231, almacenamiento en tarjeta microSD y comunicación inalámbrica; un nodo esclavo de movimiento con sensor inercial MPU6050 (acelerómetro y giroscopio 6-DOF); y un nodo esclavo con pantalla circular táctil GC9A01 de 1,28" y sensor QMI8658. Adicionalmente, se documenta una herramienta de diagnóstico que identifica el registro WHO_AM_I del MPU6050 y una utilidad de obtención de dirección MAC. El firmware se desarrolló con ESP-IDF v6.0, FreeRTOS y gestión de energía con escalado dinámico de frecuencia. Los resultados muestran transmisión estable a 100 ms (IMU) y 50 ms (touch/IMU-S3), con registro CSV local y visualización en tiempo real.

Palabras clave — Wearable, AVD, ESP32, ESP-NOW, MPU6050, QMI8658, FreeRTOS, ESP-IDF, tarjeta SD, I2C, pantalla táctil, frecuencia cardíaca.

Abstract — This paper describes the design and implementation of a distributed embedded system for monitoring Activities of Daily Living (ADL), developed as an academic project at Universidad del Cauca. The system adopts a three-node ESP32 architecture interconnected via the ESP-NOW protocol: a master node integrating a heart-rate sensor (ADC), DS3231 real-time clock, microSD storage, and wireless communication; a motion slave node with an MPU6050 inertial sensor (6-DOF accelerometer and gyroscope); and a slave node with a 1.28" GC9A01 circular touch display and QMI8658 sensor. Additionally, a diagnostic tool that identifies the MPU6050 WHO_AM_I register and a MAC address utility are documented. Firmware was developed with ESP-IDF v6.0, FreeRTOS, and dynamic frequency scaling power management. Results show stable transmission at 100 ms (IMU node) and 50 ms (touch/IMU-S3 node), with local CSV logging and real-time display.

Keywords — Wearable, ADL, ESP32, ESP-NOW, MPU6050, QMI8658, FreeRTOS, ESP-IDF, SD card, I2C, touch display, heart rate.

1. Introducción

El envejecimiento poblacional, el aumento de las enfermedades crónicas y el interés creciente por la telemedicina han impulsado el desarrollo de sistemas wearables capaces de monitorear de forma continua y no intrusiva las Actividades de la Vida Diaria (AVD) de los pacientes [1]. Estos sistemas registran señales biométricas y biomecánicas —frecuencia cardíaca, postura, movimiento— para apoyar diagnósticos clínicos, programas de rehabilitación y asistencia a personas mayores.

En este contexto, la familia de microcontroladores ESP32 (Espressif Systems) presenta características sobresalientes para aplicaciones wearables: bajo consumo, conectividad inalámbrica nativa (WiFi + Bluetooth), múltiples interfaces de periféricos y un ecosistema de desarrollo maduro basado en ESP-IDF [3]. El protocolo ESP-NOW, en particular, permite la transmisión directa punto a punto de tramas de hasta 250 bytes entre dispositivos ESP32 sin necesidad de un router intermedio, con latencias del orden de los milisegundos y un consumo energético significativamente menor al de WiFi convencional [4].

El presente artículo documenta el diseño completo de un sistema wearable distribuido para monitoreo de AVD, publicado como proyecto de código abierto en GitHub [8]. El sistema comprende cuatro submódulos funcionales: (1) un nodo maestro encargado de centra-

lizar datos, sincronizar la hora con un RTC y registrar en SD; (2) un nodo esclavo de movimiento basado en MPU6050; (3) un nodo esclavo con pantalla circular táctil y sensor QMI8658; y (4) herramientas de apoyo para diagnóstico de hardware.

1.1. Objetivos

- Diseñar una arquitectura distribuida de tres nodos ESP32 interconectados por ESP-NOW para captura y centralización de datos de AVD.
- Implementar firmware con FreeRTOS, gestión de energía y comunicación robusta en cada nodo.
- Documentar el código fuente, esquemas de conexión y resultados para su reproducibilidad pública en GitHub.

2. Estado del Arte

El reconocimiento de AVD mediante sistemas inerciales es un campo ampliamente estudiado. Anguita *et al.* [2] publicaron un *dataset* de referencia con acelerómetro de teléfono inteligente y 30 sujetos, logrando tasas de clasificación >96% con SVM para seis clases (caminar, subir escaleras, sentarse, acostarse, estar de pie, tumbarse). Chen *et al.* [1] revisaron sistemáticamente más de 200 trabajos en reconocimiento de actividad basado en sensores, identificando IMUs de bajo coste, fusión sensorial y clasificación automática como las tendencias dominantes para aplicaciones de monitoreo de

salud ambiental y asistida.

El ESP32 se ha posicionado como plataforma de referencia para sistemas wearables académicos e industriales de bajo coste. Kolban [7] documenta en detalle la arquitectura dual-core, el sistema de gestión de energía con sus modos activo, modem-sleep, light-sleep, deep-sleep y hibernación, y la pila de red WiFi/BT. El protocolo ESP-NOW permite la transmisión directa punto a punto de tramas de hasta 250 bytes entre ESP32 sin router intermedio, con latencias de 2–5 ms y un consumo significativamente menor al de WiFi convencional [4].

La combinación de ESP-NOW con FreeRTOS y gestión dinámica de frecuencia (DFS) ha sido reportada como estrategia efectiva para extender la vida útil de la batería sin comprometer la tasa de muestreo en aplicaciones de redes de área corporal (BAN). El presente trabajo adopta esta arquitectura e implementa todos sus componentes con el nivel de detalle requerido para su reproducción pública mediante un repositorio GitHub de acceso abierto [8].

Respecto al sensor MPU6050 [5], su uso masivo en el mercado de componentes ha generado un problema documentado de clones con identificadores WHO_AM_I distintos al estándar 0x68. Este trabajo aborda dicha problemática mediante una herramienta de diagnóstico dedicada que se integra al flujo de puesta en marcha del hardware, contribuyendo a la trazabilidad experimental.

La pantalla circular GC9A01 de 1,28" ha emergido como opción atractiva para wearables compactos por su bajo consumo, resolución 240×240 px y soporte para renderizado eficiente por SPI a 40 MHz. La combinación con el ESP32-S3 y la IMU QMI8658 en módulos integrados abre nuevas posibilidades para interfaces de usuario en monitoreo biomédico.

3. Arquitectura del Sistema

El sistema adopta una topología estrella con un nodo maestro y dos nodos esclavos. La Figura 1 ilustra la arquitectura general.

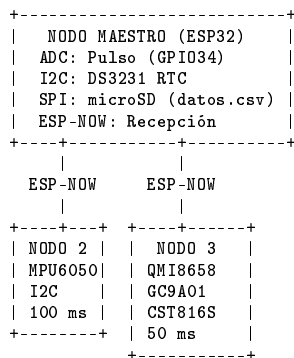


Figura 1: Arquitectura distribuida del sistema wearable AVD

La Tabla 1 resume las características principales de cada nodo.

Tabla 1: Resumen de nodos del sistema

Nodo	MCU	Función principal
Maestro	ESP32	Centraliza datos, RTC, SD, pulso
Esclavo 2	ESP32	IMU MPU6050, roll/pitch, ESP-NOW
Esclavo 3	ESP32-S3	IMU QMI8658, pantalla táctil, ESP-NOW
Herr.	ESP32	Diagnóstico WHO_AM_I (Arduino IDE)
MAC	ESP32	Obtención de dirección MAC WiFi

3.1. Protocolo de Comunicación ESP-NOW

ESP-NOW opera sobre la capa física 802.11 en modo ad-hoc sin asociación a un punto de acceso [4]. Los paquetes tienen una carga útil máxima de 250 bytes y viajan con confirmación de entrega (ACK de capa 2). En este sistema, todos los nodos utilizan la misma estructura de 32 bytes (8 floats) para los datos IMU:

```

typedef struct {
    float roll, pitch;           // Angulos (grados)
    float acc_x, acc_y, acc_z;   // Aceleracion (g)
    float gyr_x, gyr_y, gyr_z;   // Giro (deg/s)
} datos_imu_t; // 8 x 4 bytes = 32 bytes exactos

```

Listing 1: Estructura de datos transmitida por ESP-NOW (32 bytes)

La consistencia del tamaño se verifica en tiempo de compilación con `_Static_assert(sizeof(...) == 32, ...)`, garantizando la compatibilidad entre nodos incluso frente a cambios futuros.

4. Diseño e Implementación

4.1. Nodo Maestro (esp32_1_hr_rstc_sd_master)

El nodo maestro es el controlador central del sistema. Ejecuta múltiples tareas FreeRTOS concurrentes bajo un único mutex para proteger el contenedor de datos compartidos (`log_completo_t`).

4.1.1. Sensor de Frecuencia Cardíaca

La frecuencia cardíaca se adquiere localmente mediante el ADC del ESP32 en el canal 6 (GPIO 34). Se implementa un filtro en dos etapas:

1. Ráfaga de 8 muestras con media recortada (se descartan máximo y mínimo).
2. Filtro de media exponencial (EMA con $\alpha = 0,85$) sobre la señal resultante, seguido de eliminación de componente DC con una segunda EMA ($\alpha = 0,98$) como línea de base.

Esta cadena de procesamiento reduce el ruido eléctrico inherente a la lectura analógica sobre el bus de alimentación del ESP32.

4.1.2. Reloj de Tiempo Real DS3231

El DS3231 se comunica vía I2C (GPIO 21 SDA, GPIO 22 SCL, dirección 0x68) a 100 kHz. Los registros de tiempo se leen en formato BCD y se convierten a `struct tm`. El firmware permite sincronización de hora desde consola mediante el comando `SETTIME` por puerto serie.

4.1.3. Registro en microSD

Los datos se acumulan en un buffer RAM de 60 filas y se vuelcan cada minuto al archivo `/sdcard/datos.csv` mediante el sistema de archivos

FAT (ESP-VFS-FAT). La interfaz SPI se configura en el slot SPI3 (GPIO 23 MOSI, GPIO 19 MISO, GPIO 18 SCK, GPIO 5 CS). Esta estrategia reduce el número de operaciones de escritura en flash y mejora la vida útil de la tarjeta.

4.1.4. Gestión de Energía

El maestro implementa escalado dinámico de frecuencia (DFS): CPU a 240 MHz en modo activo, 80 MHz en modo normal y 10 MHz en modo inactivo. Adicionalmente se habilita *WiFi Power Save* (WIFI_PS_MIN_MODEM, ahorro ~20 mA) y *Light Sleep* con fallback si el hardware no lo soporta. El log en consola se reduce a cada 10 s para minimizar el overhead de UART.

4.2. Nodo Esclavo 2 — Movimiento (esp32_2)

El nodo de movimiento adquiere datos del sensor MPU6050 cada 100 ms y los envía al maestro cada 500 ms mediante ESP-NOW. Utiliza dos tareas FreeRTOS protegidas por un mutex binario:

- **Tarea de muestreo:** lee los 14 bytes del MPU6050 (acelerómetro + giroscopio + temperatura) en una única transacción I2C, aplica los offsets de calibración y calcula roll/pitch.
- **Tarea de envío:** copia atómicamente los datos con el mutex y los transmite por ESP-NOW.

4.2.1. Driver MPU6050 (mpu6050.c/h)

Se implementó un driver C completo y documentado con Doxygen para el MPU6050 usando el nuevo driver I2C master de ESP-IDF v5+ (driver/i2c_master.h). Sus funciones principales son:

Tabla 2: API del driver MPU6050

Función	Descripción
<code>mpu6050_init()</code>	Inicializa I2C, verifica WHO_AM_I, configura rangos
<code>mpu6050_calibrate()</code>	Promedia N muestras, calcula y almacena offsets
<code>mpu6050_read_accel_raw()</code>	Lee 3 ejes del acelerómetro (6 bytes)
<code>mpu6050_read_all_raw()</code>	Lee los 14 bytes completos (accel + temp + gyro)

La función `mpu6050_init()` acepta los valores 0x68, 0x69 y 0x72 en el registro WHO_AM_I (dirección 0x75), cubriendo tanto el MPU6050 original como variantes de silicio detectadas durante el desarrollo.

4.2.2. Calibración del MPU6050

La calibración del sensor se realiza automáticamente al arrancar el Nodo 2, mediante la función `mpu6050_calibrate()`, que toma 100 muestras con el sensor estático y calcula los offsets de cada eje:

$$\text{offset}_i = \frac{1}{N} \sum_{k=1}^N \text{raw}_{i,k} \quad (i \in \{a_x, a_y, a_z, g_x, g_y, g_z\}) \quad (1)$$

El eje a_z se corrige restando el equivalente a 1 g en LSB (16384 para rango ± 2 g) para que el reposo sobre superficie plana reporte $a_z = 1$ g y no 0 g.

Los offsets quedan almacenados en la estructura `mpu6050_t` y se aplican automáticamente en cada llama-

dada a `mpu6050_read_all_raw()`, sin necesidad de post-procesamiento en la aplicación.

4.2.3. Robustez en la Comunicación I2C

El driver implementa reintentos automáticos con hasta 5 intentos y 5 ms de espera entre cada uno para lecturas de registros. Para la lectura del WHO_AM_I en la inicialización se permite hasta 20 reintentos, dado que algunos clones del sensor requieren más tiempo para responder tras el reset. Esta estrategia mejora significativamente la tasa de éxito en la inicialización del sistema con hardware de terceros.

$$\text{Roll} = \arctan\left(\frac{a_y}{a_z}\right) \cdot \frac{180}{\pi} \quad (2)$$

$$\text{Pitch} = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \cdot \frac{180}{\pi} \quad (3)$$

Estas fórmulas proporcionan ángulos estables en condiciones cuasi-estáticas (sin vibración) con rango ± 90 .

4.3. Nodo Esclavo 3 — Touch/Display (esp_32_s3_touch_1.28)

El tercer nodo usa un ESP32-S3 con pantalla circular GC9A01 de 1,28" (240×240 px, SPI a 40 MHz) y sensor táctil capacitivo CST816S. Su función dual es: (a) visualizar un nivel de burbuja animado en tiempo real; y (b) transmitir datos de movimiento al maestro por ESP-NOW a 50 ms (20 Hz). El nodo también mide el voltaje de batería mediante un divisor resistivo en el ADC.

4.3.1. Sensor IMU QMI8658

El QMI8658 es la IMU interna del módulo ESP32-S3 Wearable 1.28", con interfaz I2C en GPIO 6 (SDA) y GPIO 7 (SCL) a 400 kHz (dirección 0x6B). Sus rangos configurados son ± 8 g para el acelerómetro (4096 LSB/g) y ± 512 °/s para el giroscopio (64 LSB/(°/s)).

4.3.2. Filtro Complementario

Para estimar los ángulos con mayor precisión que las ecuaciones 2 y 3, se aplica un filtro complementario:

$$\theta_k = \alpha_g(\theta_{k-1} + \omega \Delta t) + \alpha_a \theta_{\text{acc}} \quad (4)$$

donde $\alpha_g = 0,98$ (peso del giroscopio) y $\alpha_a = 0,02$ (peso del acelerómetro), con período $\Delta t = 50$ ms. Este filtro corrige la deriva del giroscopio usando la inclinación de largo plazo del acelerómetro.

4.3.3. Nivel de Burbuja Visual

La posición de la burbuja en pantalla se mapea directamente a los ángulos roll/pitch según: $x_b = 120 + \text{roll} \cdot k$, $y_b = 120 - \text{pitch} \cdot k$, donde k es un factor de escala de píxeles por grado. La pantalla se apaga automáticamente tras 10 s sin interacción táctil para ahorrar energía.

4.4. FreeRTOS y Sincronización entre Tareas

La arquitectura de software de cada nodo se basa en FreeRTOS con múltiples tareas concurrentes protegidas por mutexes. La Tabla 3 resume las tareas de cada nodo.

Tabla 3: Tareas FreeRTOS por nodo

Nodo	Tarea	Función
Maestro	task_rx_espnow	Recepción paquetes ESP-NOW
Maestro	task_rtc	Lectura periódica del DS3231
Maestro	task_adc_pulso	Adquisición pulso + filtrado
Maestro	task_sd_log	Volcado CSV a microSD
Nodo 2	tarea_muestreo	Lectura MPU6050 cada 100 ms
Nodo 2	tarea_envio	Envío ESP-NOW cada 500 ms
Nodo 3	tarea_imu	Lectura QMI8658 + filtro comp.
Nodo 3	tarea_espnow	Envío paquete cada 50 ms
Nodo 3	tarea_display	Renderizado pantalla GC9A01

El patrón de sincronización es consistente en todos los nodos: la tarea productora (lectura del sensor) adquiere el mutex, actualiza el buffer compartido y libera el mutex; la tarea consumidora (envío o registro) también adquiere el mutex, copia los datos y los procesa de forma independiente. Este desacoplamiento evita que una operación lenta (p.ej. escritura en SD o transmisión ESP-NOW) bloquee la adquisición de datos.

4.5. Estructura del Repositorio GitHub

La Tabla 4 documenta la estructura de carpetas del repositorio y la función de cada módulo.

Tabla 4: Estructura del repositorio GitHub

Directorio	Contenido
esp32_1_hr_rstc_sd_master/	Nodo maestro: RTC, SD, pulso, ESP-NOW
esp32_2/	Nodo IMU: MPU6050, driver C, ESP-NOW
esp_32_s3_touch_1.28/	Nodo touch: QMI8658, GC9A01, CST816S
Direccion_mac/	Utilidad de lectura de MAC WiFi
herramienta_who_i_am_*/	Diagnóstico WHO_AM_I (Arduino IDE)

Cada directorio de proyecto ESP-IDF contiene CMakeLists.txt, Doxyfile para generación de documentación automática, .devcontainer/ para entorno Docker reproducible, y .vscode/ con configuración de depuración remota por OpenOCD.

Durante la fase de integración del Nodo 2, se detectó que la biblioteca de producción rechazaba el MPU6050 por discrepancia en el registro WHO_AM_I. Se desarrolló una herramienta mínima en Arduino IDE que lee directamente el registro 0x75 vía I2C y reporta el valor real del silicio por el monitor serie a 115 200 baudios. El valor obtenido fue 0x72, que difiere del 0x68 estándar del datasheet. Esta información se incorporó al driver de producción mpu6050.c como condición de aceptación.

4.6. Utilidad de Dirección MAC (Direccion_mac)

Para configurar correctamente la tabla de peers de ESP-NOW, se desarrolló una utilidad independiente en ESP-IDF que inicializa WiFi en modo STA y reporta la dirección MAC de 6 bytes del ESP32 por el puerto serie:

```
uint8_t mac[6];
esp_wifi_get_mac(WIFI_IF_STA, mac);
printf("MAC: %02x:%02x:%02x:%02x:%02x:%02x\n",
       mac[0], mac[1], mac[2],
```

```
mac[3], mac[4], mac[5]);
```

Listing 2: Lectura de MAC WiFi en ESP-IDF

Las MACs obtenidas se usaron para configurar los arrays mac_esclavo_mpu y mac_esclavo_touch en el firmware del nodo maestro.

5. Resultados

5.1. Configuración Global de Hardware

La Tabla 5 resume la configuración de pines de todos los nodos del sistema.

Tabla 5: Configuración de hardware de los nodos

Nodo	Interfaz	Pines	Periférico
Maestro	I2C	GPIO21/22	DS3231 RTC
Maestro	SPI3	23/19/18/5	MicroSD
Maestro	ADC1-CH6	GPIO34	Pulso (HR)
Nodo 2	I2C	GPIO21/22	MPU6050
Nodo 3	I2C	GPIO6/7	QMI8658, CST816S
Nodo 3	SPI2	11/10/9/8	GC9A01 display
Nodo 3	ADC	CH0	Batería

5.2. Transmisión ESP-NOW y Tasa de Muestreo

La Tabla 6 resume las tasas de muestreo y transmisión configuradas en los nodos esclavos y la política de detección de desconexión del maestro.

Tabla 6: Rendimiento de comunicación del sistema

Nodo	Muestreo	Envío ESP-NOW	Timeout
Nodo 2	100 ms	500 ms	1000 ms
Nodo 3 (MPU6050)	50 ms	50 ms	1000 ms
Nodo 3 (S3 Touch)	—	—	60 s/bloque
Maestro (log SD)	—	—	—

El maestro detecta desconexión de cualquier nodo si no se recibe un paquete válido en más de 1000 ms, activando las banderas mpu_desconectado y touch_desconectado que se reflejan en el log CSV.

5.3. Gestión de Energía

La Tabla 7 muestra el impacto de las optimizaciones energéticas implementadas.

Tabla 7: Configuración de gestión de energía

Técnica	Configuración / Ahorro
DFS (maestro)	240→80→10 MHz según carga
WiFi Power Save	WIFI_PS_MIN_MODEM (≈20 mA ahorro)
Light Sleep	Fallback si no soportado
TX power ESP-NOW	10 dBm (40 unidades)
Pantalla (nodo 3)	Apagado auto tras 10 s sin táctil

5.4. Formato de Registro CSV

Cada fila del archivo datos.csv en la tarjeta SD contiene un registro temporal completo con granularidad de 1 segundo:

```
timestamp, pulso_raw,
roll_mpu, pitch_mpu, ax_mpu, ay_mpu, az_mpu,
gx_mpu, gy_mpu, gz_mpu,
roll_tch, pitch_tch, ...,
estado_mpu, estado_touch
```

Listing 3: Cabecera del archivo datos.csv en la tarjeta SD

Los campos `estado_mpu` y `estado_touch` indican si los nodos esclavos estaban conectados en el instante del registro. El buffer en RAM de 60 filas se vacía al disco cada 60 s, reduciendo el desgaste de la tarjeta SD y el overhead de escritura en comparación con un volcado fila a fila.

5.5. Herramienta WHO_AM_I: Resultado Experimental

La herramienta de diagnóstico (herramienta_who_i_am_id_interno_arduino_ide.ino) arrojó el siguiente resultado al ejecutarse sobre el hardware del Nodo 2:

```
Leyendo WHO_AM_I...
WHO_AM_I: 0x72
```

Listing 4: Salida del Monitor Serie de la herramienta diagnóstica

La Tabla 8 clasifica los posibles resultados.

Tabla 8: Diagnóstico por valor del registro WHO_AM_I (0x75)

Valor	Estado	Acción recomendada
0x72	OK	Hardware del proyecto; driver acepta este valor
0x68	OK	MPU6050 original; sin cambios en driver
0x98	ALERTA	Revisar máscara del driver
—	ERROR	Sin respuesta; verificar cableado

6. Discusión

6.1. Decisiones de Diseño Arquitectónico

La elección de una topología estrella con ESP-NOW frente a alternativas como BLE mesh o Zigbee se justifica por tres razones: (a) los módulos ESP32 ya incluyen el transceptor WiFi necesario sin costo adicional; (b) ESP-NOW opera en la misma banda de 2,4 GHz sin requerir un gateway de red; y (c) la implementación en ESP-IDF es directa y bien documentada con ejemplos de código de referencia.

La separación de responsabilidades entre el Nodo 2 (movimiento de tronco, MPU6050) y el Nodo 3 (interfaz táctil, QMI8658) permite una modularidad que facilita el reemplazo o actualización de cualquier nodo sin afectar los demás. La compatibilidad de la estructura `datos_imu_t` entre nodos —garantizada por `_Static_assert`— es un ejemplo de diseño defensivo que previene errores silenciosos de compatibilidad de datos.

6.2. Limitaciones del Sistema

1. **Sensor de pulso:** La lectura analógica por ADC es susceptible a ruido eléctrico y movimiento (artefactos de movimiento). Para uso clínico se recomendaría un sensor dedicado con interfaz I2C (p.ej. MAX30102).
2. **Sincronización temporal:** La sincronización del RTC requiere intervención manual por consola. Un módulo NTP sobre WiFi proporcionaría sincronización automática.
3. **Clasificación:** El sistema actual solo captura y registra datos; no implementa clasificación automática de AVD en tiempo real.
4. **Alcance ESP-NOW:** El alcance típico en interiores es de 50–100 m en línea de visión directa, lo que

puede ser insuficiente en entornos hospitalarios amplios.

6.3. Reproducibilidad y Código Abierto

Todo el código fuente, incluyendo los drivers, las configuraciones de CMake, los Doxyfiles de documentación y los binarios precompilados en formato ELF y BIN, se encuentra disponible en el repositorio GitHub [8]. Cada carpeta de proyecto contiene su propio `CMakeLists.txt` compatible con ESP-IDF v6.0 y su `devcontainer.json` para desarrollo en contenedores Docker, facilitando la reproducción del entorno de compilación en cualquier sistema operativo.

6.4. Comparativa de Sensores IMU

La Tabla 9 compara las dos IMUs utilizadas en el sistema.

Tabla 9: Comparativa entre MPU6050 y QMI8658

Parámetro	MPU6050	QMI8658
Ejes	6-DOF	6-DOF
Interfaz	I2C/SPI	I2C/SPI
Aceler. rango	±2–16 g	±2–8 g
Giro rango	±250–2000 °/s	±16–512 °/s
ADC resolución	16 bits	16 bits
Dirección I2C	0x68/0x69	0x6A/0x6B
Nodo	Esclavo 2	Esclavo 3

7. Conclusiones

1. Se diseñó e implementó un sistema wearable distribuido de tres nodos ESP32 para monitoreo de AVD, con comunicación inalámbrica por ESP-NOW, adquisición de datos IMU, frecuencia cardíaca, RTC y registro en tarjeta SD.
2. La arquitectura maestro-esclavo con ESP-NOW proporciona latencias del orden de los milisegundos con bajo overhead de firmware, y la topología estrella simplifica la sincronización y el registro centralizado.
3. El driver MPU6050 en C para ESP-IDF v5+, con soporte de calibración y variantes de silicio (`WHO_AM_I = 0x72`), constituye un componente reutilizable y documentado para proyectos de instrumentación inercial.
4. El Nodo 3 demuestra la viabilidad de integrar IMU, pantalla circular táctil y comunicación inalámbrica en un único módulo ESP32-S3 de factor de forma compacto, con filtro complementario para estimación de ángulos de alta calidad.
5. El código fuente completo, esquemas de conexión, binarios compilados y herramientas de diagnóstico están publicados en el repositorio GitHub [8], promoviendo la reproducibilidad y la transferencia tecnológica.

7.1. Trabajo Futuro

- Integrar algoritmos de clasificación de AVD (SVM, LSTM) en el nodo maestro aprovechando los 520 KB de SRAM del ESP32.
- Añadir transmisión a nube (MQTT/HTTPS) para monitoreo remoto en tiempo real.
- Implementar sincronización de tiempo NTP con

fallback al RTC DS3231 para timestamping de alta precisión.

- Explorar reducción del consumo con ESP32-C6 y protocolo IEEE 802.15.4 (Thread/Zigbee) para redes de sensores corporales de mayor densidad.

Referencias

- [1] L. Chen, J. Hoey, C. D. Nugent, D. J. Cook y Z. Yu, “Sensor-based activity recognition,” *IEEE Trans. Syst., Man, Cybern. Part C*, vol. 42, no. 6, pp. 790–808, Nov. 2012, doi: 10.1109/TSMCC.2012.2198883.
- [2] D. Anguita, A. Ghio, L. Oneto, X. Parra y J. L. Reyes-Ortiz, “Human activity recognition on smartphones using a multiclass hardware-friendly support vector machine,” en *Proc. IWAAL*, Vitoria-Gasteiz, 2012, pp. 216–223.
- [3] Espressif Systems, *ESP32 Technical Reference Manual*, v5.1, Shanghai, 2023. [En línea]. Disp.: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [4] Espressif Systems, *ESP-NOW User Guide*, v2.0, ESP-IDF Programming Guide, 2024. [En línea]. Disp.: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- [5] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification*, Rev. 3.4, doc. PS-MPU-6000A-00, Sunnyvale, CA, 2013. [En línea]. Disp.: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- [6] NXP Semiconductors, *I2C-bus specification and user manual*, Rev. 7.0, UM10204, 2021. [En línea]. Disp.: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [7] N. Kolban, *Kolban’s Book on ESP32*, Leanpub, 2017. [En línea]. Disp.: <https://leanpub.com/kolban-ESP32>
- [8] C. D. Cuellar Antury, “Sistemas Embebidos — Actividades de la Vida Diaria,” Repositorio GitHub, Univ. del Cauca, 2024. [En línea]. Disp.: <https://github.com/Cdcuellar/Sistemas-embebidos---Actividades-de-la-vida-diaria>